

sv-tpu-core

Systolic Array Matrix Multiply Unit

with Full UVM + JasperGold Formal Verification

Final Project Plan | Jul 1 – Aug 10, 2026

4-Person Team | Cadence Xcelium + JasperGold | UVM · RAL · SVA · Formal · Perf · Pipeline

A parameterized, weight-stationary systolic array matrix multiply unit, controlled via an AXI-Lite interface with a UVM RAL register model, verified by a full UVM environment with constrained-random stimulus, cross-coverage closure, error injection, three JasperGold formal proofs (accumulator overflow impossibility, AXI-Lite protocol compliance, and 2x2 functional correctness for all int8 input combinations), and a performance verification layer asserting cycle-count targets and throughput.

Badge	Owner / Type	Meaning
RTL	RTL guy (primary)	RTL design tasks
UVM	Atharva + Samarth	UVM environment tasks
VERIF	Samarth (primary)	Scoreboard, coverage, reference model, adversarial seqs
FORMAL	ATHARVA (JasperGold trained)	JasperGold formal proof tasks
PERF	JAD (perf + pipeline)	Cycle-count assertions, throughput, perf report, PE pipeline
SAMARTH	Samarth (added features)	Latency checker, weight-poison & reset-stress sequences
ALL	Whole team together	Whole-team tasks
WARN	—	Checkpoint or hard deadline
RISK	—	Risk callout — read carefully
SHIP	—	Deliverable milestone
NEW	—	Addition vs original plan

1. What We Are Building

The Problem

When a neural network runs, the vast majority of compute is matrix multiplication: $C = A \times B$, where A is input data and B is learned weights. A CPU handles it poorly. A GPU handles it reasonably. A TPU is purpose-built for it. The bottleneck on conventional hardware is memory bandwidth — cores constantly read and write intermediate results to shared memory. A systolic array eliminates this by moving data through a grid of processors rather than back and forth to memory.

Our Design — Locked Decisions

- Architecture: Weight-stationary systolic array. Weights loaded into PEs once, held stationary. Activations flow left-to-right. Results accumulate inside each PE, drain to output buffer on done.
- Array size: 4x4 PEs physically. Parameterized — change one constant to get 8x8 with zero RTL changes. Runtime DIM_REG selects active NxN subset ($N \leq 4$).
- Data types: int8 weights x int8 activations → int32 accumulator. Matches Google TPU v1. int32 prevents overflow (worst case: $4 \times 127 \times 127 = 64,516$ — far below int32 max).
- Control interface: AXI-Lite slave — three registers: DIM_REG (write), CTRL_REG (write/start), STATUS_REG (read-only/done). AXI-Lite handles control only, not data.

AXI-Lite Register Map

Register	Direction	Width	Purpose
DIM_REG	Write	3-bit	Set matrix dimension N (1–4) for this computation
CTRL_REG	Write	1-bit	Bit 0 = start. Triggers weight load then activation flow.
STATUS_REG	Read-only	1-bit	Bit 0 = done. Poll until 1 before reading output buffer.

Verification Stack — Our Differentiator

Most student projects have no verification story. Ours has six verification layers, plus a pipelined RTL datapath:

Layer	What it is	Why it matters
UVM Environment	Full UVM env: two agents, RAL register model, scoreboard, cross-coverage	Industry-standard methodology used at every semiconductor company
Python Reference Model	numpy int8 matmul — source of truth for scoreboard	Scoreboard checks every output against software ground truth
SVA Assertions	Bound to RTL via bind: AXI handshake, FSM transitions, pe_clear timing	Catches protocol and control bugs simulation may miss
JasperGold Formal	3 proofs: overflow impossibility, AXI compliance, 2x2 correctness	Proves properties hold for ALL inputs — impossible by simulation
Performance Verification	mmu_perf_checker.sv + perf_sequences.sv: cycle-count + throughput assertions	Proves the chip hits timing targets — senior-level mindset, rare at student level
Pipelined PE Datapath	One register stage in pe.sv; latency contract updated $2N-1 \rightarrow 2N$	Demonstrates the latency/throughput tradeoff — a near-universal RTL interview topic
Adversarial Verification	Latency checker (scoreboard) + weight-poison + reset-stress sequences	Targets silent data corruption and reset-recovery bugs — a true break-the-DUT mindset

2. JasperGold Formal Verification Plan

Owner: Atharva (verification lead) — only team member with JasperGold training. Every property below proves something simulation cannot prove regardless of how many tests are run. That distinction is Atharva's key interview talking point.

"We used formal verification for properties where exhaustive simulation is mathematically impossible. Each proof holds for all possible input combinations, not just the ones we thought to generate."

Proof 1 — Accumulator Overflow Impossibility

Formally proves the int32 accumulator can never overflow given int8 inputs across 4 accumulations. Worst case: $4 \times (127 \times 127) = 64,516$. int32 max: ~2.1 billion. Overflow is impossible by construction — but only formal can prove it holds for every combination.

Bound to: `pe.sv` Expected result: PROVEN Timeline: Jul 28 – Aug 1

Proof 2 — AXI-Lite Protocol Compliance

Formally proves the AXI-Lite slave correctly implements the protocol under any possible master behavior — not just the well-behaved UVM stimulus. Three properties: VALID stays asserted until READY; STATUS_REG never changes on a write; BVALID never fires without a prior write request.

Bound to: `axi_lite_slave.sv` Expected result: PROVEN x3 Timeline: Aug 1 – Aug 3

Proof 3 — 2x2 Functional Correctness [Crown Jewel]

Formally proves that for every possible combination of int8 inputs, the 2x2 subarray produces a result equal to the mathematical definition of matrix multiplication. Not "matches the reference model in simulation" — proven correct for all 2^{32} possible input combinations. This is what tape-out sign-off looks like.

Bound to: `mmu_top.sv (dim=2)` Expected result: PROVEN Timeline: Aug 3 – Aug 10

Fallback Ladder for Proof 3 (if state space is too large):

- Option 1 — k-induction: Proves property holds for all states reachable within k steps. Still a formal result. Resume line: "bounded formal correctness proof via k-induction."
- Option 2 — 1x1 proof: Formally prove a single PE produces the correct MAC result for all int8 inputs. Cleaner state space, still meaningful.
- Option 3 — Document honestly: If UNKNOWN after full effort, write up the attempt in the README. Shows formal maturity.

2B. Performance Verification Plan

Owner: Jad — this layer does not exist in any other student project at this level. Performance verification answers a different question than functional verification: not just whether the chip gets the right answer, but whether it gets the right answer in the right number of cycles.

"We verified not just functional correctness but performance correctness — every computation is asserted to complete in exactly the expected cycle count, and back-to-back throughput is measured and reported against the theoretical maximum."

The Key Property — 2N Cycle Latency (pipelined)

For a weight-stationary systolic array with dimension N, an **unpipelined** matrix multiply completes in $2N-1$ cycles. The PE pipeline register (Section 2C) adds one cycle, so the **locked contract for this project is 2N cycles**. `mmu_perf_checker.sv` asserts 2N for every computation.

Matrix size	Unpipelined (2N-1)	Pipelined contract (2N)
N=1 (1x1)	1 cycle	2 cycles
N=2 (2x2)	3 cycles	4 cycles
N=3 (3x3)	5 cycles	6 cycles
N=4 (4x4)	7 cycles	8 cycles — worst case, most important

Files Jad Owns

File	Layer	Description
<code>mmu_perf_checker.sv</code>	Perf — Checker	Signal-level SVA: asserts every NxN computation completes in exactly 2N cycles (pipelined) after activation flow. Fires the exact cycle latency is wrong. Also asserts back-to-back throughput matches theoretical maximum.
<code>perf_sequences.sv</code>	Perf — Stimulus	UVM sequences that deliberately measure throughput. Runs 10+ back-to-back computations, records cycle counts per dimension. Includes a stress sequence at N=4 for worst-case timing.
<code>perf_report.md</code>	Perf — Docs	Written after integration. Documents actual vs expected cycle counts for every dimension. Throughput table plus explanation of any deviations.

Folder Structure

```
perf/  
  mmu_perf_checker.sv  <- cycle-count assertions bound to mmu_top  
  perf_sequences.sv    <- UVM throughput measurement sequences  
docs/  
  perf_report.md       <- actual vs expected results table
```

Why This Is a 9/10 Talking Point

Performance verification is not about difficulty — it is about mindset. Most student projects verify correctness only. Professional engineers at NVIDIA verify performance too, because chips must hit clock targets or they do not ship. Owning this layer signals senior-level thinking that is genuinely rare at internship-candidate level.

2C. Pipelined PE Datapath

Owner: Jad — a design + verification addition that upgrades the RTL story from "I built a processing element" to "I designed a pipelined datapath and verified the timing tradeoff." Pipelining is one of the most fundamental concepts in chip design and is a near-universal RTL interview topic.

"We pipelined the PE datapath by inserting one register stage between the multiply-accumulate and the output. Throughput per stage improves because each stage does less work per cycle; latency increases by one cycle. The controller timing and the performance checker were both updated to reflect the new 2N cycle contract."

What Changes in the RTL

- pe.sv: insert one pipeline register (flip-flop) between the MAC result and the PE output. The result now appears one clock cycle later than in the unpipelined design.
- mmu_controller.sv: the FSM must account for the extra pipeline cycle when deciding when DONE asserts. This is a one-cycle shift in the ACTIVATION_FLOW → DONE timing.
- The latency contract changes from $2N-1$ to $2N$ cycles. Every timing assertion and the perf report must use the updated formula.

The Timing Tradeoff — Latency vs Throughput

This is the core interview talking point. Pipelining trades a small latency increase for better per-stage timing, which is exactly the decision real hardware teams make.

	Unpipelined (original)	Pipelined (this addition)
Latency ($N=4$)	7 cycles ($2N-1$)	8 cycles ($2N$)
Per-stage critical path	MAC + routing in one cycle	MAC and routing split across two cycles
Throughput	Limited by combined path	Higher — shorter critical path per stage
Interview value	"I built a PE"	"I designed a pipelined datapath and explain the tradeoff"

The Full-Circle Story (why this pairs with performance verification)

Because Jad owns both the pipeline and the performance checker, the two connect into one narrative: Jad added the pipeline register to pe.sv, which shifted the latency by one cycle; the mmu_perf_checker.sv assertion caught the timing change because the old $2N-1$ formula no longer held; Jad updated the contract to $2N$ and logged the finding. Designing a change and catching its timing implication through your own verification is a genuinely senior story.

Interview answer:

"I pipelined the PE datapath and my own performance checker flagged the latency shift from $2N-1$ to $2N$. I traced it to the new register stage, confirmed it was expected, and updated the timing contract — design and verification of the same change."

2D. Latency Verification — Ownership Clarification

Two team members verify the computation latency. This is **intentional** — defense in depth — but the overlap must be coordinated so the two checks never disagree. This section is the single source of truth for how latency is verified and who owns what.

The one shared number: latency is 2N (pipelined).

The unpipelined array would finish in 2N-1 cycles. Jad's PE pipeline register (Section 2C) adds exactly one cycle, so the locked contract for the whole project is **2N**. Samarth's scoreboard latency checker and Jad's perf-checker SVA must **both** assert 2N — not 2N-1. If either still uses 2N-1 after the pipeline lands, it will fire a false failure. This is the most likely integration mistake on the project; lock 2N in the spec doc in P1.

Two checks, two abstraction levels — no duplicated work

Check	Owner	Level	What it does / why it is distinct
latency_checker (in mmu_scoreboard.sv)	Samarth	Transaction-level	Records the cycle CTRL_REG start fires and the cycle STATUS_REG done asserts; asserts observed_latency == 2N. Confirms the result arrives at the right time, as part of the scoreboard's per-transaction check.
mmu_perf_checker.sv (bound to mmu_top)	Jad	Signal-level (SVA)	A bind-based SVA assertion that fires on the exact cycle a 2N violation occurs — better waveform debug. Also measures back-to-back throughput, which the scoreboard check does not do.

Division of responsibility (so nobody steps on anybody)

- Samarth owns the **latency property in the scoreboard** (transaction-level pass/fail) and his two adversarial sequences (weight-poison, reset-stress).
- Jad owns the **signal-level latency SVA** (mmu_perf_checker.sv) plus everything throughput-related (perf_sequences.sv, perf_report.md) and the PE pipeline that sets the 2N number.
- Atharva owns the formal proofs and the UVM environment the two checks run inside. Atharva's formal AXI/overflow proofs do not touch latency — no overlap there.
- In interviews each person describes their own level: Samarth “transaction-level latency in the scoreboard,” Jad “signal-level latency SVA plus throughput,” Atharva “formal correctness.” Same property, three honest distinct stories.

Why keep both instead of deleting one?

Real silicon teams verify critical timing at multiple levels on purpose — a scoreboard catches it at the transaction boundary, an assertion pin-points the exact cycle. Keeping both, and being able to explain the difference, reads as thoroughness, not redundancy.

3. Module Hierarchy

Module / File	Layer	Owner	Description
mmu_top.sv	RTL — Top	RTL	Wrapper. Instantiates all RTL sub-modules. DUT boundary.
axi_lite_slave.sv	RTL — Interface	RTL	AXI-Lite register file. Decodes DIM_REG, CTRL_REG, STATUS_REG (read-only enforced).
mmu_controller.sv	RTL — Control	RTL	FSM: IDLE → WEIGHT_LOAD → PE_CLEAR → ACTIVATION_FLOW → DONE. DONE timing accounts for the PE pipeline stage (2N contract).
systolic_array.sv	RTL — Datapath	RTL	NxN generate block of pe.sv. Parameterized on N. Fans pe_clear to all PEs.
skew_buffer.sv	RTL — Setup	SAMARTH OPTIONAL	OPTIONAL (P4 bonus): Systolic Data Setup block. Parameterized shift-register cascade — row K delayed K cycles — staggering activations before systolic_array.sv. Moves skew from the testbench driver into RTL, as in TPU v1 Fig. 1. ~20-30 lines.
pe.sv	RTL — Leaf	RTL	Single PE. One MAC + one pipeline register stage (Jad). pe_clear zeros accumulator in exactly one cycle. Output registered → +1 cycle latency.
output_buffer.sv	RTL — Output	RTL	Holds int32 results until testbench reads them after done.
mmu_if.sv	Interface	ALL	SV interface bundle. Must be agreed before either track writes code.
tb_top.sv	TB — Top	ALL	Testbench top. Instantiates DUT + UVM env + all bind statements.
mmu_env.sv	TB — Env	UVM	UVM environment. Both agents + scoreboard + RAL model.
mmu_reg_model.sv	TB — RAL	VERIF	UVM RAL register model. DIM_REG (RW), CTRL_REG (RW), STATUS_REG (RO).
axi_agent.sv	TB — Agent	UVM	AXI-Lite master agent. Drives transactions via RAL model.
data_agent.sv	TB — Agent	UVM	Data agent. Drives activations, monitors output buffer.
mmu_scoreboard.sv	TB — Check	VERIF	Compares DUT outputs vs Python golden model. Negative checking for error injection. Hosts Samarth's transaction-level latency_checker (2N).
mmu_coverage.sv	TB — Coverage	VERIF	Cross coverage: dim x weight_pattern x activation_pattern bins. 90%+ target.
mmu_sequences.sv	TB — Stimulus	UVM	Constrained-random + directed + error injection + back-to-back + reset. Hosts Samarth's weight_poison_seq and reset_stress_seq.
axi_lite_sva.sv	SVA — Protocol	UVM	AXI handshake simulation assertions. Bound to axi_lite_slave via bind.
mmu_controller_sva.sv	SVA — FSM	VERIF	FSM transition + pe_clear timing assertions. Bound to mmu_controller.
pe_sva.sv	SVA — PE	VERIF	PE-level simulation assertions (zero-in → zero-out). Bound to pe.
pe_formal.sv	Formal — PE	FORMAL	Overflow impossibility property. Run in JasperGold, not simulation.
axi_formal.sv	Formal — AXI	FORMAL	AXI protocol compliance properties. Run in JasperGold.
mmu_formal.sv	Formal — Top	FORMAL	2x2 functional correctness property. Run in JasperGold.
mmu_perf_checker.sv	Perf — Checker	PERF	Signal-level SVA: asserts 2N cycle latency (pipelined) per computation. Throughput assertions for back-to-back runs. Bound to mmu_top. See 2D for split vs Samarth.
perf_sequences.sv	Perf — Stimulus	PERF	UVM sequences measuring throughput and cycle counts across all matrix dimensions.
ref_model.py	Golden Model	VERIF	Python numpy matmul. Source of truth for scoreboard.
Makefile	Infra	ALL	make regress → compile + elaborate + simulate + coverage + perf check.
BUGS.md	Docs	ALL	Bug log. Every integration bug: symptom, test, root cause, fix.
latency_checker (in mmu_scoreboard.sv)	TB — Check	SAMARTH	Transaction-level latency verification: asserts DONE fires exactly 2N cycles after START. Pairs with Jad's signal-level SVA (see 2D).
weight_poison_seq (in mmu_sequences.sv)	TB — Stimulus	SAMARTH	Adversarial weight matrices (all-zero, all-127, identity, checkerboard) targeting silent datapath corruption.
reset_stress_seq (in mmu_sequences.sv)	TB — Stimulus	SAMARTH	Reset asserted at every FSM state individually. Verifies clean recovery and correct next-computation result.
perf_report.md	Docs	PERF	Actual vs expected cycle counts for all dimensions. Throughput table.

Light-purple rows are Jad's performance + pipeline additions; lavender rows are Samarth's adversarial additions (integrated into existing files). mmu_if.sv must be agreed before either track writes drivers or RTL port lists. Formal .sv files run only through JasperGold — never through Xcelium. skew_buffer.sv is an OPTIONAL P4 bonus (see timeline + rule 18).

4. Project Timeline: Jul 1 – Aug 10

P0 Pre-production — Foundations & Setup | Now → Jul 1

ALL	Read the TPU v1 paper (arxiv 1704.04760) — architecture section only. 20 minutes. Do it this week.
ALL	Create GitHub repo. Folder structure: /rtl, /sva, /formal, /perf, /tb, /uvm, /docs, /scripts. Agree on branch strategy.
ALL	Sketch the 4x4 PE array on paper including pe_clear signal. Agree on all signal names before writing code.
ALL	Confirm Xcelium license access. Compile a hello-world .sv file. Fix environment issues now, not in July.
ALL	Confirm JasperGold license access. Launch the tool. Fix any license issues now.
ALL	Lock data types: int8 activations, int8 weights, int32 accumulators. This decision does not change.
WARN	Deliverable: repo live, paper read, both tools verified — by Jul 1.

P1 Architecture Week — Lock the Design Before Writing Code | Jul 1 → Jul 8

S-	ALL	Draw and agree on block diagram. Every module, every interface, every arrow. Commit as PNG to /docs.
J-	ALL	Write the spec document (2 pages max). <u>Data widths</u> , <u>timing diagram</u> , <u>AXI register map</u> , <u>pe_clear timing</u> , <u>definition of done</u> . No code until it exists.
S-	ALL	Define all port lists in the spec: PE ports (inc. pe_clear), array ports, controller ports, AXI-Lite slave ports.
A-	ALL	Write UVM test plan: list every scenario (normal NxN, error injection, back-to-back, reset). Specify coverage targets (90%+ all cross bins).
J-	ALL	Define RAL register spec in the spec doc: DIM_REG (RW, 3-bit), CTRL_REG (RW, 1-bit), STATUS_REG (RO, 1-bit). Access types and reset values locked.
A-	FORMAL	Define formal proof scope in the spec doc: three proofs, three files, owned by verification lead. Write the property statements in prose before coding them.
J-	PERF	NEW: Define performance spec in the spec doc. Write the cycle-latency formula in prose. Agree what back-to-back throughput means. Define what perf_report.md must contain before coding.
A-	PERF	NEW (pipeline): Decide in the spec whether the PE output is registered. If yes, lock the latency contract at 2N (not 2N-1). The timing diagram must show the extra pipeline cycle. Locked before any RTL is written.
S-	SAMARTH	NEW: Add the latency spec to the doc — DONE asserts exactly 2N cycles after activation flow (the pipelined contract). This single number is shared by Samarth's scoreboard checker and Jad's perf SVA. See Section 2D.
	WARN	HARD RULE: No one writes a single line of RTL or UVM until the spec doc is committed and all four have approved it.
	WARN	Deliverable: <u>spec doc</u> + <u>block diagram</u> committed by Jul 3. ¹²

Thursday 10:00 - 11:30
EDT

P2 Build Sprint — RTL and UVM Built in Parallel | Jul 8 → Jul 28

RTL Track (primary: RTL guy — others review PRs)

RTL	Week 1 (Jul 8–15): pe.sv. Single PE. MAC logic + one pipeline register stage (built with Jad). pe_clear input: synchronous accumulator zero in exactly one cycle. Smoke-test standalone with directed stimulus.
RTL	Week 2 (Jul 15–22): systolic_array.sv. NxN generate block of pe.sv. Parameterized on N. Inter-PE horizontal activation wiring. pe_clear fanned out to all PEs. J A
RTL	Week 3 (Jul 22–28): mmu_controller.sv + axi_lite_slave.sv + mmu_top.sv. FSM complete. pe_clear asserts exactly one cycle in PE_CLEAR state. STATUS_REG enforced read-only. S

UVM Track (primary: you + other verif guy — RTL guy reviews PRs)

UVM	Week 1 (Jul 8–15): UVM skeleton + RAL model. env.sv, agent stubs, driver/monitor stubs, interface file. mmu_reg_model.sv with all three registers. RAL adapter to AXI agent.
VERIF	Week 2 (Jul 15–22): Golden model + scoreboard. Write ref_model.py first — numpy int8 matmul with int32 accumulation. Verify manually. Port check logic to mmu_scoreboard.sv.
FORMAL	Week 2 (Jul 15–22): Write all three formal property files. pe_formal.sv, axi_formal.sv, mmu_formal.sv. Goal: compile clean in JasperGold. Write .tcl stub scripts. A
UVM	Week 3 (Jul 22–28): Sequences + coverage. Constrained-random stimulus. Cross coverage groups. Back-to-back (10+). Reset mid-computation. Error injection sequences.

Performance Track (primary: Jad — runs alongside UVM track)

PERF	NEW Week 2 (Jul 15–22): Hand-trace a 2x2 computation cycle by cycle against mmu_controller.sv. Confirm the pipelined 2N formula. Document expected cycle counts for N=1,2,3,4 in the spec doc.
PERF	NEW Week 1 (Jul 8–15): Pair with RTL guy on the pe.sv pipeline register. Confirm the registered output adds exactly one cycle. Update the cycle-trace to the 2N contract.
PERF	NEW Week 3 (Jul 22–28): Write mmu_perf_checker.sv — assert 2N latency (pipelined) bound to mmu_top via bind. Write perf_sequences.sv with 10+ back-to-back throughput runs. Goal: both compile clean. J

Adversarial Verification Track (primary: Samarth — inside scoreboard & sequences)

SAMARTH	NEW Week 2 (Jul 15–22): latency_checker in mmu_scoreboard.sv. Timestamp logic: record cycle when CTRL_REG start fires and when STATUS_REG done asserts. Assert observed_latency == 2N. Fail the scoreboard if off by one cycle. Runs on every transaction. make
SAMARTH	NEW Week 2 (Jul 15–22): weight_poison_seq in mmu_sequences.sv. Four adversarial weight matrices: all-zeros (result must be zero), all-127 (worst-case accumulation), identity (result == activations), checkerboard. Checked via the existing Python reference model — no new checker needed.
SAMARTH	NEW Week 3 (Jul 22–28): reset_stress_seq in mmu_sequences.sv. Assert reset individually at WEIGHT_LOAD, PE_CLEAR, ACTIVATION_FLOW. After each: verify FSM returns to IDLE, accumulators zeroed, next computation correct. Systematic per-state coverage.
WARN	Checkpoint Jul 15: pe.sv simulating correctly + UVM skeleton + RAL model compiling clean.
WARN	Checkpoint Jul 22: Full array RTL + scoreboard + latency_checker + all three formal .sv files compiling + perf cycle-count analysis done.
WARN	Checkpoint Jul 28: All RTL + all UVM + all formal TCL + mmu_perf_checker.sv + perf_sequences.sv + weight_poison_seq + reset_stress_seq written and compiling. Ready to integrate.

UVM Integration

ALL	Wire UVM environment to mmu_top DUT. Connect interface signals. Activate bind statements. Compile first, simulate second.
UVM	Run RAL built-in sequences first: uvm_reg_hw_reset_seq and uvm_reg_access_seq. Fix any register model or AXI slave bugs before directed tests.
ALL	Run directed tests first. Hand-compute a 2x2 result. Drive it. Verify scoreboard catches it. Fix structural bugs before adding randomness.
UVM	Verify SVA assertions fire on deliberately illegal stimulus (error injection) and do NOT fire on legal stimulus. Log any false-positive fires in BUGS.md.
ALL	Run random tests. Debug scoreboard mismatches via waveforms. This is where you live.
UVM	Run back-to-back sequences (10+ transactions). Watch for state retention bugs — accumulator values leaking between runs. Log in BUGS.md.
UVM	Run reset sequences. Assert reset mid-computation. Verify FSM recovers and next transaction produces correct result.
UVM	Run error injection sequences. Verify negative scoreboard checks pass: no spurious output during illegal operation window.
VERIF	Coverage closure. Check cross coverage report. Write targeted sequences to hit uncovered bins. Target: 90%+ all cross bins.

Performance Verification (Jad)

PERF	NEW: Run perf_sequences.sv against the integrated DUT. Verify mmu_perf_checker.sv assertions pass on all legal stimulus for N=1 through N=4.
PERF	NEW: Verify mmu_perf_checker.sv fires correctly on a deliberately injected cycle-count violation. Confirm the checker actually catches timing bugs, not just that it compiles.
PERF	NEW: Collect actual cycle counts for every matrix dimension. Compare to expected 2N (pipelined). Log any deviations in BUGS.md immediately with root cause.
PERF	NEW (pipeline): Confirm the registered PE output shifts latency by exactly one cycle vs the unpipelined trace. This is the documented latency/throughput tradeoff — capture it for the perf report and README.

Adversarial Verification (Samarth)

SAMARTH	NEW: Run weight_poison_seq. Verify the scoreboard catches any datapath error for all four adversarial matrices. A passing all-zeros test with nonzero output is a silent data-corruption bug. Log findings in BUGS.md.
SAMARTH	NEW: Run reset_stress_seq. Verify clean FSM recovery at every state. Log any accumulator leakage or wrong post-reset computation in BUGS.md. High-yield bug surface — expect at least one real bug here.
SAMARTH	NEW: Verify latency_checker fires correctly — inject a one-cycle DONE delay and confirm it catches it; confirm it passes on all legal transactions. Coordinate with Jad: both checkers must agree on 2N (see 2D). An unexercised checker is not a passing checker.
ALL	Full regression. Every test must pass. Re-run 3x. Flaky tests mean a race condition exists — find and fix it.
RISK	Scoreboard mismatches from off-by-one in controller sequencing or pe_clear timing take longer than expected. Start Phase 3 the moment Jul 28 arrives — do not wait.
RISK	SVA assertion false positives can consume a day. Triage immediately — is the RTL wrong or the property wrong? Do not debug both simultaneously.
RISK	NEW: If mmu_perf_checker.sv fires on legal stimulus, check whether the controller FSM cycle count matches the 2N pipelined contract. Common causes: off-by-one in PE_CLEAR accounting, or forgetting the pipeline register cycle.

Formal Proofs — Run in JasperGold (Atharva only)

FORMAL	Jul 28 – Aug 1: Overflow Impossibility (Proof 1). Run <code>pe_overflow.tcl</code> . Set <code>int8</code> input assumptions. Expected: PROVEN. If counterexample, constraints are wrong — iterate on TCL, not SV.
FORMAL	Aug 1 – Aug 3: AXI Compliance (Proof 2). Run <code>axi_compliance.tcl</code> . If counterexample: real RTL protocol bug. Log in <code>BUGS.md</code> . Fix immediately.
FORMAL	Aug 3 – Aug 5: 2x2 Correctness (Proof 3 — first attempt). Run <code>twobytwo_correct.tcl</code> . If UNKNOWN, apply k-induction. If still inconclusive, continue into Phase 4.
WARN	Deliverable by Aug 5: clean regression (all tests pass, coverage met) + Proofs 1 and 2 PROVEN + <code>perf_checker</code> and <code>latency_checker</code> both passing on all legal stimulus.

P4 Polish & Ship — Make It Look as Good as It Works | Aug 5 → Aug 10

FORMAL	Final attempt on Proof 3 if still inconclusive. Try k-induction, then 1x1 fallback. Document outcome honestly regardless of result.
FORMAL	Commit all proof results to <code>/formal/results/</code> . Screenshot of PROVEN in JasperGold for each completed proof — design name visible. Link from README.
PERF	NEW: Finalize <code>perf_report.md</code> . Table of actual vs expected cycle counts for N=1 through N=4 (2N pipelined). Throughput for back-to-back runs. Include the pipeline latency/throughput tradeoff explanation. Any deviations explained with root cause.
PERF	NEW: Verify <code>mmu_perf_checker.sv</code> is included in <code>make regress</code> and passes cleanly on every run. Performance assertions are part of the regression, not a separate manual step.
SAMARTH	NEW: Add adversarial-verification results to the README — the 2N latency property passing across N=1..4, the weight-poison results, and the reset-stress recovery. One screenshot of a latency violation caught (from <code>BUGS.md</code>) if applicable.
ALL	Write the README. One-sentence summary. Block diagram. How to run (make regress). Formal proofs section with PROVEN screenshots. Performance results section linking <code>perf_report.md</code> .
ALL	Verify <code>make regress</code> works from a cold clone. A teammate clones fresh and runs it. If it fails, it is not done.
ALL	Capture waveform screenshots of a 4x4 multiply with annotated signals. Show one assertion firing on error injection and one <code>perf_checker</code> assertion in action. Add to <code>/docs</code> .
VERIF	Export and commit final Xcelium coverage closure report to <code>/docs/coverage_report/</code> .
ALL	Finalize <code>BUGS.md</code> . Minimum 3–5 entries including at least one <code>perf_checker</code> finding. Every entry: symptom, test, root cause, fix.
SHIP	Tag v1.0 on GitHub. Repo is interview-ready. Update resumes. Ship.

OPTIONAL Bonus — `skew_buffer.sv` (only if v1.0 is otherwise taggable)

Attempt the following ONLY if regression is clean, coverage is closed, formal proofs have run, `BUGS.md` is populated, and the README is drafted. This is a bonus, not a requirement (see Rule 18).

SAMARTH OPTIONAL	Build <code>skew_buffer.sv</code> — a parameterized shift-register cascade. Row 0: no delay; row 1: 1-cycle; row 2: 2-cycle; row K: K-cycle delay. ~20-30 lines. Instantiate in <code>mmu_top.sv</code> between the activation input and <code>systolic_array.sv</code> . This is the TPU v1 "Systolic Data Setup" block (Fig. 1 of arxiv 1704.04760), moving skew out of the testbench driver and into RTL.
SAMARTH OPTIONAL	Add one SVA assertion: after K cycles of delay, the output of row K matches the input from K cycles ago. Add one directed test: feed known values in, verify they emerge correctly staggered.
SAMARTH OPTIONAL	Update the <code>latency_checker</code> to account for the skew buffer's contribution — it adds N-1 cycles to the first result, which is already factored into the total latency contract, so the total does not change. Add one coverpoint: all N delay depths exercised.

5. Work Split & Ownership

Person	Primary Owns	Also Involved In
RTL guy	pe.sv, systolic_array.sv, mmu_controller.sv, axi_lite_slave.sv, mmu_top.sv	Reviewing UVM sequences, debug waveforms, reviewing assertion bind files, co-building PE pipeline register
Atharva (verif lead)	mmu_env.sv, axi_agent.sv, data_agent.sv, mmu_reg_model.sv, Makefile, ALL THREE FORMAL PROOFS (pe_formal.sv, axi_formal.sv, mmu_formal.sv, all .tcl scripts, all PROVEN screenshots)	RTL code reviews, integration debugging, coverage closure support, owns the UVM env both checkers run inside
Samarth (verif)	mmu_scoreboard.sv, mmu_coverage.sv, ref_model.py, mmu_controller_sva.sv, pe_sva.sv, BUGS.md; latency_checker (in scoreboard), weight_poison_seq + reset_stress_seq (in sequences)	Error injection, back-to-back & reset sequences, final regression ownership; coordinates 2N latency with Jad (2D)
Jad (perf + pipeline)	mmu_perf_checker.sv (NEW), perf_sequences.sv (NEW), perf_report.md (NEW), PE pipeline register in pe.sv (NEW, co-built with RTL guy)	Integration debugging of timing issues, BUGS.md entries for perf findings, README perf + pipeline section

6. Why This Lands at Google / NVIDIA

What they see	Why it matters
Systolic array MMU	You built the actual computational core of a TPU — not a tutorial, the real thing.
Weight-stationary architecture	You made an architectural decision and can explain the tradeoff vs output-stationary.
int8 x int8 → int32	Matched TPU v1 quantized arithmetic. Can explain the power efficiency rationale.
AXI-Lite control interface	Real-world SoC awareness. Every serious accelerator has a control bus.
Parameterized (N=4 scalable to N=8)	RTL maturity — you know how to write reusable, scalable hardware.
Full UVM environment	Industry-standard methodology. Most student projects have no verification story.
UVM RAL register model	Signals complete UVM methodology knowledge. Direct screening question at NVIDIA/AMD.
SVA assertions bound to RTL	Methodology of professional verification engineers. Most student projects have zero.
Error injection + neg. scoreboard	You tried to break the DUT, not just confirm happy paths. Shows verification mindset.
Cross functional coverage closure	Proves meaningful scenario combinations were exercised, not just toggle coverage.
Overflow impossibility — PROVEN	Mathematical guarantee, not a simulation result. Impossible to fake.
AXI compliance — PROVEN	Protocol correctness under all master behaviors, not just your testbench stimulus.
2x2 functional correctness — PROVEN	Correct for all 2^{32} int8 input combinations. No simulation campaign can claim this.
Performance verification — 2N proven	Cycle-count + throughput assertions. Senior-level mindset. Almost nobody does this at student level.
Pipelined PE datapath	Demonstrates the latency vs throughput tradeoff — a fundamental RTL concept asked in nearly every design interview.
Cycle-accurate latency checker	Separates timing correctness from functional correctness — verified at two abstraction levels on purpose.
Weight-poisoning sequences	Targets silent data corruption under adversarial inputs. Distinct from protocol-level error injection.
Reset-at-every-FSM-state stress	Systematic reset-recovery verification across all FSM states. A high-yield bug surface.
BUGS.md with root causes	Answers "tell me about a bug you found" — the most common verfif interview question.
make regress one-command flow	Repo is professional and reproducible. Interviewers can run it themselves.
Systolic Data Setup block (skew_buffer.sv)	Implements the activation alignment block named explicitly in the TPU v1 paper — shows you read the source material and built what it describes.

7. Rules — Do Not Break These

- 1. No RTL written before the spec doc is committed and all three approve it.
- 2. No one merges to main without a PR reviewed by at least one other team member.
- 3. If you are more than 3 days behind a checkpoint, the whole team stops and recalibrates. Do not silently fall behind.
- 4. The scoreboard must be driven by the Python golden model. Do not hand-compute expected outputs.
- 5. Coverage report must show cross bin targets hit before Phase 4 begins. No skipping coverage closure.
- 6. Assertions must compile clean and fire on illegal stimulus during error injection. An assertion that never fires was never exercised.
- 7. BUGS.md is updated during integration, not after. Log bugs the moment they are found.
- 8. Formal proof files (.sv and .tcl) are run only through JasperGold — never through Xcelium.
- 9. Commit PROVEN screenshots to /formal/results/ before tagging v1.0. If the screenshot is not there, the proof is not done.
- 10. README is written by Aug 8. Not Aug 10. Not after.
- 11. make regress must work from a cold clone before tagging v1.0.
- 12. Everyone can explain every module, every assertion, every formal proof, the 2N performance property, the pipeline tradeoff, and which latency check they own (2D). You will be asked in interviews.
- 13. NEW: mmu_perf_checker.sv must be included in make regress and pass cleanly before tagging v1.0. Performance verification is not optional.
- 14. NEW: perf_report.md must contain actual vs expected cycle counts for all four matrix dimensions before tagging v1.0.
- 15. NEW (pipeline): The latency contract is 2N (pipelined PE), not 2N-1. Every timing assertion, the scoreboard latency_checker, the perf report, and the README must use 2N. Anyone can explain why the pipeline adds one cycle.
- 16. NEW (latency): Samarth's scoreboard latency_checker (transaction-level) and Jad's mmu_perf_checker.sv (signal-level) must agree on 2N and are kept on purpose as defense-in-depth. Neither is deleted; each owner explains their level (see Section 2D).
- 17. NEW (Samarth): The latency_checker must fire on a deliberately injected off-by-one during integration. weight_poison_seq and reset_stress_seq must each produce at least one BUGS.md entry. If they find no bugs, they were not run aggressively enough.
- 18. NEW (OPTIONAL): skew_buffer.sv is only attempted after v1.0 is otherwise taggable. It is a bonus, not a requirement.